This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3652009

Lightweight Federated Few-Shot Learning-Based Network Intrusion Detection for Resource-Constrained IoT Devices

Ahsan Saleem, *Student Member, IEEE,*, and Walaa Hamouda, *Senior Member, IEEE,*

***Abstract*—Resource-constrained IoT devices are particularly vulnerable to network intrusions, highlighting the need for effective security mechanisms. Network Intrusion Detection Systems (NIDS) address these threats by monitoring traffic and detecting malicious behavior. However, deploying anomaly-based NIDS on such devices faces three key challenges: privacy concerns in centralized training, the scarcity of labeled data and high computational demands of machine learning models in real-world IoT environments. To address these challenges, this study presents a Lightweight Federated Few-Shot Learning (Light-FFSL) framework, specifically designed for secure and efficient intrusion detection in IoT settings. The framework preserves data privacy by employing federated learning, which enables collaborative model training across devices without exposing sensitive data. To overcome data scarcity, an LSTM-based prototypical few-shot model is used, allowing the system to generalize from only a small number of labeled samples per class. To reduce computational cost, the framework incorporates dynamic scheduling and convergence-based updates. These mechanisms dynamically select participating devices and reduce unnecessary communication for federated training. In addition, the model architecture integrates pruning and post-training quantization into an LSTM-driven prototypical network. Pruning removes redundant parameters before training to reduce complexity, while quantization converts the trained model into an efficient integer-based form, lowering memory requirements and speeding up inference. The effectiveness of the proposed framework is validated through experiments on real-world IoT network datasets, demonstrating its ability to operate efficiently under resource constraints.**

***Index Terms*—Network Intrusion Detection (NIDS), Federated Learning (FL), Few-Shot Learning (FSL), IoT Networks, Pruning and Quantization.**

I. Introduction

The rapid proliferation of resource-constrained devices is driven by technological advancements and the growing demand for energy-efficient, compact solutions tailored to modern applications [1]. These IoT devices, recognized as transformative technologies, enable seamless connectivity, automation, and data-driven decision-making across diverse domains. They are increasingly deployed in critical sectors such as healthcare, agriculture, transportation, and smart infrastructure [2]. Their deployment helps optimize operations and enhances service delivery. Industrial forecasts predict that the number of IoT devices globally will surpass 40 billion by 2030 [3]. This growth underscores their widespread adoption and integration into everyday life. These devices significantly improve operational efficiency and resource utilization by facilitating real-time data collection, processing, and analysis. Moreover, the ability of IoT devices to deliver actionable insights and drive technological advancement establishes them as key enablers of intelligent and interconnected ecosystems [4]. However, the extensive integration of IoT devices into diverse applications has also introduced significant security vulnerabilities, particularly network intrusions.

IoT networks, characterized by their heterogeneity and constrained computational resources, are inherently susceptible to sophisticated network attacks that exploit systemic weaknesses [5]. The consequences of such intrusions are extensive, including unauthorized access, data breaches, service disruptions, and the potential compromise of critical infrastructure. These incidents not only reduce the operational efficiency of IoT networks but also erode user confidence and result in substantial financial losses [6]. Network intrusion detections play a crucial role in mitigating these risks. By monitoring network traffic, identifying intruders, and providing early detection of malicious activities, Network Intrusion Detection System (NIDS) enables timely and effective countermeasures [7]. NIDS significantly strengthens the resilience, security, and reliability of IoT systems, ensuring their sustained and secure functionality.

NIDS employs various detection techniques to identify and mitigate cyber threats within IoT environments. The two primary approaches to intrusion detection are signature-based detection and anomaly-based detection [8]. Signature-based intrusion detection [9] operates by comparing incoming traffic threats against a database of predefined attack signatures, enabling the identification of known threats. This method is effective in detecting previously documented intrusions. However, it lacks adaptability to emerging threats, particularly zero-day attacks and novel attack techniques, making it less suitable for dynamic and evolving IoT ecosystems. In contrast, anomaly-based detection identifies intrusions by analyzing deviations from normal network behavior rather than relying on predefined attack signatures. This approach is particularly advantageous in IoT environments, where attack patterns frequently evolve. To enhance anomaly-based detection, various Machine Learning (ML) techniques [10] have been widely explored, including supervised learning methods, unsupervised learning approaches, and Deep Learning (DL) models. These models analyze network traffic patterns, learn complex behaviors, and identify subtle deviations that may

The authors are with the Department of Electrical and Computer Engineering, Concordia University, Montreal, QC H3G 1M8, Canada (e-mail: ah_eem@live.concordia.ca; hamouda@ece.concordia.ca)

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 23,2026 at 15:13:11 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3652009

indicate malicious activity. As a result, ML-driven anomaly detection significantly enhances the efficiency and security of NIDS, making them better suited for evolving network attacks. Recent studies have shown that the increasing intelligence of IoT demands joint optimization of learning, communication, and security to ensure resilience in dynamic environments [11] [12]. These works underscore the importance of adaptive and resource-efficient coordination, motivating the development of intrusion detection frameworks that maintain robustness under similar constraints.

While ML has significantly enhanced the capabilities of anomaly-based NIDS, its deployment in IoT environments presents several challenges [13], [14]. A primary concern is the computational overhead associated with high-complexity, AI-based NIDS. Given that many IoT devices have limited computational power, memory, and energy, deploying such resource-intensive solutions is impractical without significant optimization [13]. In addition, data privacy during the training phase remains a critical issue. IoT devices frequently generate sensitive information, and the prevalent reliance on centralized training increases the risk of data breaches and unauthorized access [14]. Furthermore, the scarcity of labeled data poses a substantial challenge, as ML models generally require large, annotated datasets that are often unattainable in real-world IoT scenarios [15]. These limitations highlight the need for lightweight, ML-driven approaches that balance efficiency, privacy, and adaptability, ensuring the feasibility of intrusion detection in resource-constrained IoT environments.

A. Related Work

Despite notable advancements and the widespread adoption of deep learning [16] in IoT networks, deploying NIDS in resource-constrained IoT environments remains a significant challenge [17] due to computational overhead, data privacy concerns, and limited availability of labeled data. Gyamfi et al. [18] proposed a lightweight NIDS combining One-class Support Vector Data Description (OI-SVDD) and Extreme Learning Machine (AS-ELM) with Multi-access Edge Computing (MEC) support to offload computations, while Zhao et al. [19] introduced a neural network-based NIDS with Principal Component Analysis (PCA)-based feature reduction.Yang et al. proposed NIDS-CNNRF [20], which integrates CNN for feature extraction and Random Forest for classification, with PCA for redundancy removal and ADASYN for data imbalance, improving detection efficiency in IoT networks. Similarly, the GMCWAE [21] framework combines Gaussian mixture priors with Wasserstein autoencoders to extract compact yet expressive features, reducing computational burden in constrained IoT environments. However, while these approaches improve efficiency, they do not incorporate privacy-preserving mechanisms nor address the scarcity of labeled data, making them incomplete for IoT deployments.

Tong and Zhang [22] addressed label scarcity via self-supervised learning; however, their method lacks privacy-preserving mechanisms. Similarly, FS-IDS [23] introduced a CNN-based few-shot learning model that achieved high detection accuracy in SCADA networks, yet did not support privacy-preserving, limiting its suitability for privacy-sensitive IoT environments. Zha et al. introduced A-NIDS [24], an adaptive IDS that uses clustering with stacked CTGANs to generate synthetic traffic and mitigate catastrophic forgetting. Wu et al. proposed TCG-IDS [25], a temporal contrastive graph-learning model that learns robust temporal embeddings in a self-supervised manner, reducing reliance on large labeled datasets. While both methods effectively address the challenge of limited labeled data, they remain computationally heavy and lack integration with federated privacy-preserving mechanisms, which limits their applicability to resource-constrained IoT environments.

FL-based NIDS offers improved privacy. Friha et al. [26] presented Federated Learning-based Intrusion Detection System (FELIDS) for agricultural IoT, and Bhale et al. [27] developed Optimum Placement Strategy to Mitigate mixed-rate DDoS attack in IoT System (OPTIMIST) for mixed-rate DDoS mitigation using LSTM and Generative Adversarial Network (GAN)-generated data. While effective, both approaches still depend on labeled datasets. FL enhancements via knowledge distillation [28] and transfer learning [29] improve adaptability but increase computational demands, limiting suitability for low-power devices. Benameur et al. [30] and Aljuhani et al. [31] offer compressed or explainable FL-based NIDS, but label efficiency remains a concern.

Semi- and self-supervised FL-based models such as FL-SSAL (Federated Learning-Empowered Semi-Supervised Active Learning) [32], and Semi-supervised Spatiotemporal DL model (SS-Deep-ID [33] address labeled data scarcity but remain resource-intensive. Yao et al. [34] proposed a Packet-level Adversarial Traffic Generation (PATG) framework to evaluate the robustness of deep learning-based NIDS under realistic adversarial traffic in Industrial IoT. In addition, an interaction-aware trust management scheme combines ML-based intrusion detection with device reputation scoring to mitigate on–off and bad-mouthing attacks in IoT systems [35]. Although these works enhance robustness and privacy in adversarial and trust-aware contexts, they still overlook resource-efficiency and the data scarcity dimension, which are critical in IoT deployments.

In summary, while existing approaches address parts of the problem space, few simultaneously ensure lightweight design, robust privacy preservation, and effective learning from limited labeled data, highlighting the need for integrated solutions.

B. Motivation and Contribution

As IoT devices continue to operate in distributed and often unsupervised environments, the need for timely and reliable intrusion detection becomes increasingly important. However, their limited processing power, memory, and energy present serious challenges for deploying conventional machine learning models. In addition, privacy concerns arise when sensitive data must be transmitted for centralized training, making decentralized approaches more desirable. Complicating matters further, labeled examples of network attacks are often limited in real-world deployments, reducing the effectiveness of fully supervised learning. These factors together highlight the need

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 23,2026 at 15:13:11 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3652009

for a lightweight, privacy-preserving intrusion detection solution that can work effectively with limited labeled data and operate within the strict resource constraints typical of IoT devices.

The main contributions of this work can be summarized as follows:

- Privacy and Resource-Aware Federated Learning: We develop an FL–based intrusion detection framework that employs dynamic device scheduling and convergence-based updates. This design reduces communication overhead, ensures fairness across heterogeneous IoT devices, and preserves the privacy of locally generated network data.
- Few-shot learning for data efficiency: To address the scarcity of labeled attack data in real-world IoT environments. The LSTM encoder captures temporal patterns in traffic, while the prototypical layer forms class embeddings from few labeled samples. During inference, unseen attacks are identified by measuring Euclidean distance to existing prototypes, enabling rapid adaptation without retraining. This allows the system to generalize from only a few labeled samples per class and effectively detect both frequent and emerging attack types.
- Lightweight deployment: We incorporate structured pruning and integer quantization to minimize model complexity. Pruning removes redundant LSTM units and low-magnitude weights, while quantization converts weights and activations to 8-bit integers for low-power inference. Together, these techniques reduce memory footprint and accelerate inference, enabling efficient on-device deployment.
- Comprehensive evaluation: We validate the proposed framework through extensive experiments on multiple real-world IoT intrusion detection datasets. The evaluation includes comparisons with state-of-the-art baselines, analysis of communication and computation efficiency, and robustness assessments under varying device and network conditions.

II. System Model and Problem Formulation

A. System Model

We consider an IoT network environment, as shown in Fig. 1, which comprises a set of devices:

$$D_i = \{D_1, D_2, \ldots, D_{N_i}\}, \quad i \in \{1, 2, \ldots, M\}. \tag{1}$$

Each device D_{ij} generates network traffic over time t, represented as:

$$x_{ij}(t) = \{p_1, p_2, \ldots, p_k\}, \quad p_k \in \mathbb{R}^d \tag{2}$$

where p_k represents the k-th network parameter (e.g., packet size, protocol type, timestamps), and d is the feature space dimensionality. The traffic data at a given time step t follows a probabilistic distribution:

$$X_{ij}(t) \sim P_{x_{ij}}, \quad \forall t \in T \tag{3}$$

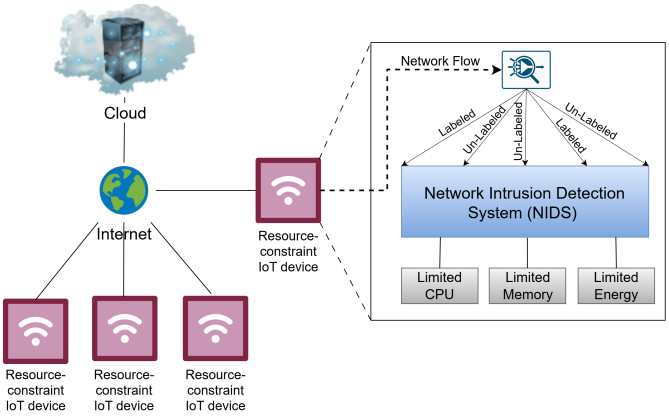


Fig. 1: System Model

where $P_{x_{ij}}$ represents the underlying distribution of network traffic features. The real-time data stream $X_{ij}(t)$ is used for intrusion detection.

Each device is equipped with anomaly-based NIDS that processes network traffic $x_{ij}(t)$ and assigns a classification score:

$$s_{ij}(t) = f_{\theta_{ij}}(x_{ij}(t)) \tag{4}$$

where $f_{\theta_{ij}}$ is the detection function parameterized by θ_{ij}. The classification decision follows a general decision rule:

$$y_{ij}(t) = g(s_{ij}(t), \tau_{ij}) \tag{5}$$

where $g(\cdot)$ is a general function mapping the detection score $s_{ij}(t)$ to a decision label based on a threshold τ_{ij}. The decision space can be defined as:

$$y_{ij}(t) \in \mathcal{Y}, \quad \mathcal{Y} = \{0, 1, \ldots, K\} \tag{6}$$

where K represents the number of possible attack classes in a multi-class setting.

We consider that each IoT device operates under constrained computational resources, meaning that each device has limited resources available for running NIDS. To model these limitations, we define a resource vector for each device as follows:

$$R_{ij} = \{CPU_{ij}, MEM_{ij}, EN_{ij}\}, \quad CPU_{ij}, MEM_{ij}, EN_{ij} \in \mathbb{R}^+ \tag{7}$$

where CPU_{ij} denotes the available processing power, MEM_{ij} represents the available memory, and EN_{ij} is the energy budget. Each device executes local computations based on available resources. The computational complexity of intrusion detection on each device is modeled as:

$$C_{ij} = \mathcal{O}(n_s \cdot d) \tag{8}$$

where n_s represents the number of processed network packets, and d is the feature dimensionality. The power consumption for intrusion detection is defined as:

$$P_{ij} = \alpha \cdot CPU_{ij} + \beta \cdot MEM_{ij} \tag{9}$$

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 23,2026 at 15:13:11 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3652009

where α, β are energy consumption coefficients.

The training of NIDS must take data privacy into consideration, ensuring that sensitive network traffic information is not exposed during the learning process. To preserve the confidentiality of network traffic data, all training must be conducted locally on the device without exposing sensitive inputs. The system enforces a local data protection requirement that ensures outbound information during processing or communication reveals no meaningful content from the original traffic data. This is quantified as:

$$\mathbb{E}[(\mathcal{I}_{ij}(t))] \leq \epsilon, \quad \forall t \tag{10}$$

where $(\mathcal{I}_{ij}(t))$ quantifies the amount of private information that could be exposed during processing or communication, and ϵ represents an upper bound on allowable information leakage.

To train the NIDS model, we consider that the dataset $\mathcal{T}$ consists of both labeled and unlabeled samples, represented as:

$$\mathcal{T} = \mathcal{T}_L \cup \mathcal{T}_U, \quad \mathcal{T}_L \ll \mathcal{T}_U \tag{11}$$

where $\mathcal{T}_L$ is the labeled subset and $\mathcal{T}_U$ is the unlabeled subset. Given the nature of evolving cyber threats, the labeled dataset is extremely small, such that, $N_L = |\mathcal{T}_L| \ll |\mathcal{T}_U| = N_U$, highlighting the severe scarcity of labeled instances. Additionally, the attack classes can be divided into seen attacks (K_{seen}) with available labeled data and novel attacks (K_{novel}) with little to no labeled data, where $|K_{novel}| \ll |K_{seen}|$.

B. Problem Formulation

Given the system model, our objective is to optimize the performance of a NIDS in an IoT environment while adhering to three fundamental constraints: resource limitations, data privacy preservation, and limited labeled data availability.

We define the optimization problem to maximize the detection performance of the NIDS across all IoT devices while minimizing computational cost and energy consumption. The detection performance can be quantified as a function of classification accuracy, computational complexity, and energy efficiency:

$$\max_{\theta} \sum_{i=1}^{M} \sum_{j=1}^{n_i} \left[\lambda_1 \cdot Acc_{ij}(\theta) - \lambda_2 \cdot C_{ij} \quad -\lambda_3 \cdot P_{ij}\right] \tag{12}$$

where $Acc_{ij}(\theta)$ represents the detection accuracy of the NIDS model $f_{\theta_{ij}}$ deployed on device D_{ij}, C_{ij} denotes the computational complexity of intrusion detection on D_{ij}, P_{ij} represents the power consumption of the NIDS, $\lambda_1, \lambda_2, \lambda_3$ are weighting parameters to balance detection accuracy, computational efficiency, and energy consumption.

To ensure practical feasibility in IoT environments, the optimization problem is subject to the following constraints:

1) Resource Constraints: To ensure computational feasibility, we introduce a normalized resource utilization function that ensures no device exceeds its allocated budget:

$$\frac{1}{CPU_i^{\max}} \sum_{j=1}^{n_i} CPU_{ij} \leq 1, \tag{13}$$

$$\frac{1}{MEM_i^{\max}} \sum_{j=1}^{n_i} MEM_{ij} \leq 1, \tag{14}$$

$$\frac{1}{EN_i^{\max}} \sum_{j=1}^{n_i} EN_{ij} \leq 1, \quad \forall i \in \{1, 2, \ldots, M\} \tag{15}$$

This ensures that the total consumption of processing power, memory, and energy do not exceed the predefined system limits for each IoT device.

2) Privacy Constraint: To ensure data privacy during model training, we restrict the model from exposing sensitive network traffic information. We model this constraint as:

$$u_{ij}(t) \cap x_{ij}(t) = \emptyset, \quad \forall t \tag{16}$$

where $x_{ij}(t)$ represents the raw network traffic data on device D_{ij} at time t, $u_{ij}(t)$ denotes the information transmitted externally from device D_{ij} at time t. The condition $u_{ij}(t) \cap x_{ij}(t) = \emptyset$ ensures that outbound communication contains no raw data, samples, or any direct or invertible transformation thereof.

3) Labeled Data Constraint: Due to the scarcity of labeled data, we impose a constraint on the labeled-to-unlabeled ratio rather than absolute quantities:

$$\frac{N_L}{N_U} \leq \delta, \quad \delta \ll 1 \tag{17}$$

where δ represents the fraction of labeled data compared to the total available data, which remains significantly small. This constraint highlights the challenge of limited supervision in training deep learning models for NIDS.

These formulations ensures that the NIDS optimally balances intrusion detection accuracy, computational efficiency, and energy consumption while ensuring privacy preservation and effective learning under severe data scarcity. The constraints reflect the real-world challenges of deploying NIDS in resource-constrained IoT environments, privacy concerns are critical, evolving network attacks, and computational power is limited.

C. Threat Model

We categorize the threat model into two categories: frequent threats that the NIDS has been trained to recognize, and new/evolving threats that are uncommon, novel, and unseen in training.

1) Frequent Attacks: Let $\mathcal{Y}_{\text{freq}} = \{1, \ldots, k_f\}$ denote the set of frequent attack classes, and use 0 to represent benign traffic. The training dataset is then given by $\mathcal{D}_{\text{train}} = \{(x_j, y_j)\}_{j=1}^{n}$, where each feature vector $x_j \in \mathbb{R}^d$ represents a traffic sample and its corresponding label y_j belongs to $\{0\} \cup \mathcal{Y}_{\text{freq}}$.

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 23,2026 at 15:13:11 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3652009

2) New and Evolving Attacks: Let $\mathcal{Y}_{\text{new}}$ denote the set of attack classes that are absent during training, with $\mathcal{Y}_{\text{new}} \cap \mathcal{Y}_{\text{freq}} = \varnothing$. The NIDS model is required to detect such novel attacks and, given only a small number of labeled samples, efficiently extend the operational label set. This reflects practical conditions in which the threat landscape continuously evolves, and rare or emerging attacks may not be represented in the training data, with only a few labeled samples available for adaptation.

III. Light-FFSL-based NIDS

A. Framework Overview

In this section, we present a Light-FFSL framework tailored for network intrusion detection in privacy-aware and resource-constrained IoT environments. The proposed framework is designed to address three critical challenges: (1) preserving data privacy during model training, (2) enabling learning from limited supervision, and (3) ensuring computational efficiency on resource-constraint devices. To ensure data privacy, the framework adopts the FL paradigm, which enables decentralized model training by keeping sensitive data on local devices. This approach significantly reduces the privacy risks associated with centralized data aggregation. To address the limited availability of labeled data, FSL is employed, allowing the model to generalize effectively from a minimal number of labeled examples. To enhance computational efficiency, the framework incorporates three key strategies: dynamic scheduling, convergence-based federated updates, and model pruning and quantization. Dynamic scheduling mechanisms select participating IoT devices for federated training based on their computational capacity, ensuring balanced workload distribution. Convergence-based update strategies further reduce communication overhead by initiating global model aggregation only when meaningful performance improvements are observed. Additionally, a pruned and quantized LSTM-based prototypical FSL model is implemented to reduce model complexity and memory usage, enabling efficient deployment on resource-constrained IoT devices. An illustrative Fig. **??** provides an overview of the proposed Light-FFSL-based NIDS framework. The subsequent sections detail the core components and mechanisms of the proposed approach.

B. Federated Learning

FL is a decentralized ML paradigm that enables multiple IoT devices to collaboratively train a shared model without exchanging raw data [36]. Unlike traditional centralized learning, where sensitive data is transmitted to a central server for processing, FL ensures data privacy by keeping training data localized on individual devices. This approach is particularly beneficial in resource-constrained IoT environments, where privacy regulations and limited communication bandwidth make centralized learning impractical.

In the proposed framework, FL is employed to train the intrusion detection model across distributed IoT devices while mitigating privacy risks. The training process follows the standard Federated Averaging (FedAvg) approach [37], which consists of four main steps: local model training, model aggregation, global model distribution, and secure aggregation to ensure that only the aggregated update is revealed to the server without exposing individual contributions.

- *Local Model Training*: Each participating IoT device i trains a local model w_i using its private dataset T_i. The local objective function is defined as:

$$F_i(w) = \frac{1}{|T_i|} \sum_{j \epsilon T_i} \mathcal{L}(w; x_j, y_j) \tag{18}$$

  where $\mathcal{L}(w; x_j, y_j)$ represents the loss function for input-output pairs (x_j, y_j).
- *Model Aggregation*: The locally trained models are sent to a central aggregator, which computes the global model update using a weighted average:

$$w_{t+1} = \sum_{i=1}^{N} \frac{|D_i|}{\sum_j |D_j|} w_i^t \tag{19}$$

  where N is the number of participating devices, and w_i^t is the locally trained model at round t.
- *Global Model Distribution*: The updated global model w_{t+1} is then redistributed to the IoT devices for the next training iteration, allowing continuous improvement of the model without requiring raw data exchange.
- *Secure Aggregation:* While FL prevents data from leaving IoT devices, individual model updates may still expose sensitive information through gradient inversion or membership inference attacks. To mitigate this risk, FedAvg can be extended with Secure Aggregation (SA). In this mechanism, each client i masks its local update before transmission as

$$\hat{\mathbf{w}}_i^t = \mathbf{w}_i^t + \mathbf{r}_i - \mathbf{r}_{i+1}, \quad \mathbf{r}_{N+1} \triangleq \mathbf{r}_1, \tag{20}$$

  where $\mathbf{w}_i^t$ is the local model update and $\hat{\mathbf{w}}_i^t$ is the masked model update of client i at round t, and $\mathbf{r}_i$ is a random mask generated by client i. Due to the circular cancellation of masks, the server only recovers

$$\sum_{i=1}^{N} \hat{\mathbf{w}}_i^t = \sum_{i=1}^{N} \mathbf{w}_i^t, \tag{21}$$

  thus concealing individual contributions while still enabling correct global model aggregation. Unlike differential privacy, which perturbs updates with noise and can degrade accuracy [38], SA maintains model utility while introducing only modest cryptographic overhead, making it practical for resource-constrained IoT devices.

As the model is trained across distributed devices, maintaining efficiency in resource-constrained environments becomes critical. Allowing all devices to participate equally in every training round can lead to rapid energy depletion, communication overhead, and imbalance in computation. Moreover, device heterogeneity in terms of processing power, availability, and energy levels can significantly impact training effectiveness. To address these challenges, the proposed Light-FFSL incorporates a dynamic scheduling and convergence-Based FL Update for FL training.

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 23,2026 at 15:13:11 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3652009

C. Resource-Aware Device Scheduling

Standard FL approaches typically assume uniform device participation, which is impractical in heterogeneous and resource-constrained IoT networks where devices differ in energy capacity, computational power, and network conditions. Frequent selection of high-performing devices accelerates convergence but quickly depletes their limited resources, while underutilizing weaker devices introduces bias and unfair workload distribution. To overcome these limitations, the proposed Light-FFSL framework integrates a dynamic scheduling mechanism that optimizes both the selection and frequency of device participation across FL rounds. Unlike the traditional top-k selection strategy, which randomly chooses devices, dynamic scheduling adaptively modulates participation based on real-time metrics including energy efficiency, latency, computational power, convergence contribution, and availability. This approach ensures fairness, sustains device longevity, and balances workload distribution. Moreover, as the model progresses toward convergence, the mechanism gradually reduces redundant updates by lowering the participation frequency of less impactful devices, thereby minimizing communication overhead and energy consumption while maintaining effective training.

1) Scoring Metrics for Device Participation: Each device is evaluated using a scoring system that reflects its suitability for participation in the current FL round. The score for each device i is computed as:

$$S_i = \Theta_1 S_E + \Theta_2 S_L + \Theta_3 S_C + \Theta_4 S_{CR} + \Theta_5 S_A \tag{22}$$

where S_E is the energy efficiency that ensures energy-constrained devices participate less frequently. S_L is a latency, which prioritizes low-latency devices to enhance real-time model updates. S_C is a computational power that accounts for the device's processing capabilities. S_{CR} is a convergence Rate, which rewards devices contributing to rapid model convergence. S_A is an availability that reflects device reliability based on historical uptime and participation.

The required metrics are obtained from device-local training statistics, such as validation loss trends and runtime system indicators, which are inherently available during model updates. As a result, the scheduling mechanism leverages existing information without introducing additional measurement or communication burden, preserving the lightweight nature of the framework.

The weights $\theta_1, \cdots, \theta_5$ can dynamically be adjusted based on system priorities and satisfy:

$$\sum_{j=1}^{5} \Theta_j = 1. \tag{23}$$

2) Probability and Frequency of Selection: Once scores are computed, the top-k highest-scoring devices are selected for full participation, while non-top devices ($i \notin K$) are probabilistically included. The probability of inclusion for top-k devices is given by:

$$P_i = \frac{S_i}{\sum_{j \in k} S_j}. \tag{24}$$

ensuring that higher-scoring devices in the top-k contribute more frequently. For non-top devices, the probability of participation is:

$$P_i = \frac{S_i}{\sum_{j \notin k} S_j}. \tag{25}$$

This ensures that lower-ranked devices are not completely excluded but rather contribute at a reduced rate based on their computed score.

Beyond selection, participation frequency is also modulated dynamically. High-scoring devices participate frequently but not at a constant rate. Their frequency is adjusted based on energy constraints and their marginal contribution to global model convergence. If a top-performing device exhibits low remaining energy, its participation frequency is automatically reduced:

$$f_{base} = f_{init} \times \left(1 - \frac{t}{T}\right) \times \left(\frac{E_i}{E_{max}}\right). \tag{26}$$

$$f_i = f_{base} \times S_E, \tag{27}$$

where f_{base} is the base participation frequency. Similarly, if a device's updates show diminishing improvement to the global model, its participation rate is decreased to reduce redundant updates. This reduction follows:

$$f_i \leftarrow f_i \times \left(1 - \frac{\Delta_{threshold}}{\Delta L_{global}}\right), \tag{28}$$

where ΔL_{global} represents the global model's convergence progress, and $\Delta_{threshold}$ is a predefined threshold indicating when training updates become less significant.

3) Convergence-Based FL Updates: To further enhance efficiency, device participation is modulated according to the global convergence trend. The participation frequency for each device is adjusted based on its local convergence rate S_{CR} and the global convergence trend:

$$f_i^{convergence} = f_i \times \left(1 - \frac{\Delta_{global}}{\Delta L_{threshold}} \times S_{CR}\right). \tag{29}$$

As the global model stabilizes, dynamic participation reduces update frequency across all devices, conserving resources, preventing redundant updates, and minimizing network congestion.

The final participation probability for each device is computed as:

$$P_i^{final} = P_i \times f_i^{convergence}, \tag{30}$$

ensuring that participation dynamically adjusts to both device conditions and model convergence trends. The **algorithm** 1 illustrates dynamic participation of devices and convergence-driven updates in FL.

Now, we discuss the dynamic participation and convergence-based FL update with an example. Consider an FL-based NIDS deployed across an IoT network with five devices (D1–D5), each assigned a participation score S_i based on energy efficiency, latency, computational power, convergence rate, and availability. In an early training round, the top-3 devices (D1, D2, D3) are selected for full participation, while D4 and D5 are probabilistically

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 23,2026 at 15:13:11 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3652009

included based on their normalized scores. Initially, each device's participation frequency is modulated by its energy level, ensuring that energy-constrained devices contribute less frequently. As training progresses, the global model loss reduction is monitored, and once it drops below a predefined threshold $\Delta_{threshold}$, the participation frequency of all devices is dynamically reduced using convergence-based update $f_i^{convergence}$. For instance, if D1 has a high convergence impact, it maintains frequent updates, whereas D3, with minimal impact, sees its updates reduced. The final participation probability is adjusted as P_i^{final}, ensuring that only the most effective updates are transmitted, thereby minimizing redundant communication and energy consumption while preserving model accuracy.

Algorithm 1 Dyanmic Scheduling and Convergence Based FL Updates

Require: N (devices), k (selection size), $\Delta_{\text{threshold}}$, f_{base}, $\theta_1, \theta_2, \theta_3, \theta_4, \theta_5$
Ensure: Participating devices in the FL round
▷ Calculate global convergence measure
$\Delta_{\text{global}} \leftarrow |L_{\text{prev}} - L_{\text{current}}|$
▷ Compute and rank scores for all devices
for $i \in N$ **do**
  $S_i \leftarrow \sum_{j=1}^{5} \theta_j S_j$
end for
Sort devices based on S_i; select top-k devices set $\mathcal{K}$
for $i \in N$ **do**
▷ Assign initial participation probabilities
  if $i \in \mathcal{K}$ **then**
    $P_i \leftarrow 1$
  else
    $P_i \leftarrow \frac{S_i}{\sum_{j \notin \mathcal{K}} S_j}$
  end if
▷ Adjust frequency based on device energy
  $f_i \leftarrow f_{\text{base}} \times S_E$
▷ Update frequency considering convergence rate
  $f_i^{\text{conv}} \leftarrow f_i \times \left(1 - \frac{\Delta_{\text{global}}}{\Delta_{\text{threshold}}} S_{CR}\right)$
▷ Calculate final probability of participation
  $P_i^{\text{final}} \leftarrow P_i \times f_i^{\text{conv}}$
▷ Decide participation based on final probability
  if $P_i^{\text{final}} > \text{Rand}(0,1)$ **then**
    Mark device i as participating
  else
    Mark device i as not participating
  end if
end for
return participating devices

D. Few-Shot Learning Model

The proposed Light-FFSL framework employs a few-shot learning model to enable effective intrusion detection under limited supervision. Specifically, it integrates a Prototypical Network with a LSTM encoder, which extracts temporal features from sequential network traffic data and constructs class prototypes from only a handful of labeled samples. Query instances are then classified based on their distance to these prototypes, allowing the model to generalize effectively to novel or sparsely represented attack classes. This design ensures adaptability to evolving threats while maintaining strong detection performance in scenarios where labeled training data is scarce. In network intrusion detection, each incoming network traffic instance consists of sequential data patterns that require temporal analysis for accurate classification. We employ an LSTM-based encoder to extract meaningful feature embeddings from these sequences, as LSTMs are well-suited for modeling long-term temporal dependencies and mitigating vanishing gradient issues that commonly limit recurrent neural networks or feed-forward models.

Given an input sequence $X = x_1, x_2, \cdots, x_T$, where T represents the sequence length, the LSTM processes each time step iteratively:

$$h_t = \sigma\left(W_h h_{t-1} + W_x x_t + b\right). \tag{31}$$

where h_t is the hidden state at time step t, W_h and W_x are weight matrices, b is the bias term, and σ represents an activation function, often a tanh or ReLU function. The final feature representation is obtained from the last hidden state:

$$f_\theta(x) = h_T. \tag{32}$$

This embedding $f_\theta(x)$ serves as the input for prototype computation in the few-shot classification stage.

1) Prototype Computation and Query Classification: In a few-shot learning setup, the dataset is divided into a support set and a query set. The support set consists of a few labeled examples per class, providing reference points for classification, while the query set contains unseen samples that need to be classified based on their similarity to the support set.

To compute class prototypes in a few-shot learning setup, we calculate the mean embedding of support set samples for each class:

$$p_c = \frac{1}{|S_c|} \sum_{x \epsilon S_c} f_\theta(x), \tag{33}$$

where p_c is the prototype representation of class, S_c is the support set for class c and $f_\theta(x)$ is the embedding of a sample x obtained from the LSTM encoder. Query samples are classified based on their Euclidean distance from

$$d\left(q, p_c\right) = \left\| f_\theta\left(q\right) - p_c \right\|^2, \tag{34}$$

where q represents the query sample. The model assigns q to the nearest prototype, allowing classification even with minimal labeled samples.

After computing class prototypes and performing query classification, the trained few-shot model undergoes compression through structured pruning and integer quantization, as described in the following section, to enable efficient deployment on resource-constrained IoT devices.

E. Model Compression

The few-shot learning model improves adaptability under limited labeled data, but its direct deployment on IoT devices can still be hindered by computational and memory

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 23,2026 at 15:13:11 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3652009

constraints. To overcome these limitations, Light-FFSL incorporates model compression techniques that preserve detection accuracy while significantly reducing resource consumption. In particular, structured pruning and post-training quantization are employed to remove redundant parameters, minimize model size, and accelerate inference.

1) Model Pruning: To reduce the computational complexity of the LSTM-based prototypical network prior to deployment, Light-FFSL applies a two-stage pruning strategy. First, we apply pre-pruning techniques that optimize the architecture before the training process begins. Instead of training a full-scale model and pruning afterward, we construct a smaller model using structured sparsity principles [39]. The process involves reducing the number of LSTM units, removing redundant neurons, and applying low-rank approximations to weight matrices:

$$W \approx U_k S_k V_k^T, \tag{35}$$

where U_k and V_k are reduced-dimensional matrices, and S_k contains the top k singular values. This transformation significantly reduces the model's complexity before training, allowing efficient deployment on resource-constrained IoT devices.

Following initial training, structured pruning is applied to further refine the model by removing weights that contribute the least to performance. Given an LSTM weight matrix W, we remove entire groups of parameters (e.g., neurons or gates) whose collective importance falls below a threshold τ :

$$\mathcal{G}_k' = \begin{cases} \mathcal{G}_k, & \text{if } \|\mathcal{G}_k\|_2 \geq \tau, \\ 0, & \text{otherwise,} \end{cases} \tag{36}$$

where $\mathcal{G}_k'$ represents the pruned group of LSTM parameters. The pruning ratio r is computed as:

$$r = \frac{\left\|W'\right\|_0}{\|W\|_0}, \tag{37}$$

where $\|W\|_0$ and $\left\|W'\right\|_0$ denote the number of nonzero parameters before and after pruning, respectively. Once pruning is completed, fine-tuning is performed to recover any accuracy loss. By combining pre-pruning and structured pruning, we achieve a highly efficient model that retains predictive performance while requiring fewer computational resources.

2) Model Quantization: To further reduce the memory footprints, we apply Full Integer Quantization [40], which is particularly suitable for resource-constrained IoT devices. This approach converts both weights and activations to integer representations, reducing the memory footprint and accelerating inference speed. Instead of performing computations in floating-point precision, the model operates entirely in integer arithmetic, making it significantly more efficient for edge devices with limited computational capacity.

Formally, full integer quantization is performed by mapping weights and activations to integer representations using scaling factors:

$$A_{quant} = round(\frac{A}{s_A}), \;\; W_{quant} = round(\frac{W}{s_W}), \tag{38}$$

where A_{quant} and W_{quant} are the quantized activation and weight values, respectively. s_A and W_A are learned scaling factors for activations and weights, respectively. This ensures that numerical precision is preserved while reducing the model's overall complexity.

Algorithm 2 illustrates the optimized FSL with pruning and quantization. The algorithm begins by taking labeled training data $D = \{(x_i, y_i)\}$, a support set S, and a query set Q, aiming to produce an optimized Prototypical Network model M. It starts with pre-pruning, where the LSTM units are reduced using low-rank decomposition via Singular Value Decomposition (SVD), approximating the weight matrix as $W \leftarrow U_k S_k V_k^T$ (Line 2) to minimize unnecessary computations before training. The model then proceeds to training, where for each batch in the dataset, it trains on support set samples (Line 7-8). For each class c in S, a class prototype is computed as $p_c \leftarrow \frac{1}{|S_c|} \sum_{x \in S_c} f_\theta(x)$ (Line 3-7), ensuring that each class is represented efficiently. After training, the algorithm applies structured pruning, iterating through all weights $W_{i,j}$ and setting those below a predefined threshold τ to zero (Line 09-13), reducing complexity while preserving accuracy. Since pruning removes parameters, the model undergoes fine-tuning to recover any performance loss. Next, post-quantization is applied, where weights and activations are converted into integer representations using $A_{quant} = round(A/s_A)$ and $W_{quant} = round(W/s_W)$ (Line 15-18), significantly improving memory efficiency and inference speed. During inference, for each query sample q, the Euclidean distance to each class prototype is computed as $d(q, p_c) = ||f_\theta(q) - p_c||^2$ (Line 22), with the query being assigned to the class with the minimum distance, computed as $y_q \leftarrow \arg\min_c d(q, p_c)$ (Line 25), ensuring few-shot classification. Finally, the optimized model is deployed on IoT devices, enabling real-time, lightweight, and energy-efficient intrusion detection, making it ideal for resource-constrained environments.

F. Complexity Analysis

The computational complexity of the proposed algorithm involves multiple steps. Initially, the complexity of the LSTM network used for feature extraction is $O(\mathcal{T} \cdot n^2)$, where T represents the sequence length of input data and n denotes the number of LSTM units. This shows that while the number of trainable parameters and the model size remain constant for different values of T, the number of computations increases linearly with T, directly impacting inference latency and energy consumption. After structured pruning, this complexity significantly reduces to $O(\mathcal{T} \cdot (r \cdot n^2)$, where $r < 1$ is the pruning ratio, effectively decreasing the computational load by eliminating redundant parameters. The computation of class prototypes for each class requires $O(|S_c| \cdot d)$, with $|S_c|$ as the number of support samples in class c, and d representing the embedding dimension of the feature space. During inference and classification, each query sample necessitates computing the Euclidean distance to each of the C class prototypes, resulting in a complexity of $O(C \cdot d)$. The post-quantization step converts weights and activations to integer values, substantially improving the model's runtime efficiency, although it

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 23,2026 at 15:13:11 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3652009

Algorithm 2 Optimized Few-Shot Learning with Pruning and Quantization

Require: Labeled training data $D = \{(x_i, y_i)\}$, support set S, query set Q
Ensure: Optimized Prototypical Network model M

▷ Pre-Pruning: Reduce model complexity via low-rank decomposition
$W \leftarrow U_k S_k V_k^T$
for each batch in training data **do**
  for each class c in S **do**
    $p_c \leftarrow \frac{1}{|S_c|} \sum_{x \in S_c} f_\theta(x)$
  end for
end for
▷ Structured Pruning: Remove neurons below threshold τ
for each parameter group $\mathcal{G}_k$ in model **do**
  if $\|\mathcal{G}_k\|_2 < \tau$ **then**
    Set $\mathcal{G}_k = 0$
  end if
end for
▷ Post-Quantization: Convert weights and activations to integer representations
for each weight W and activation A **do**
  $A_{\text{quant}} \leftarrow \text{round}(A/s_A)$
  $W_{\text{quant}} \leftarrow \text{round}(W/s_W)$
end for
▷ Inference and Classification: Predict class for each query sample
for each query sample $q \in Q$ **do**
  for each class prototype p_c **do**
    $d(q, p_c) = ||f_\theta(q) - p_c||^2$
  end for
  ▷ Assign class with closest prototype
  $y_q \leftarrow \arg\min_c d(q, p_c)$
end for

does not alter the asymptotic complexity. Therefore, the overall computational complexity of the algorithm is combined as:

$$O(\mathcal{T} \cdot (r \cdot n^2) + |S_c| \cdot d + C \cdot d). \tag{39}$$

IV. Theoretical Analysis

This section analyzes the learning dynamics of the proposed Light-FFSL model and establishes how the LSTM-based Prototypical Network converges when trained under few-shot episodes, structured pruning, integer quantization, and dynamic device participation. Federated communication acts primarily as a coordination mechanism, whereas the optimization behavior—and ultimately the convergence rate, is governed by the compressed model updates.

A. Learning Objective and Gradient Dynamics

For device i, let S_c and Q_c denote the support and query sets of class c. The local episodic few-shot objective is

$$L_i(\theta) = \frac{1}{|Q_i|} \sum_{(q, y_q) \in Q_i} \ell\big(d(q, p_{y_q}(\theta)),\, y_q\big), \tag{40}$$

$$d(q, p_c) = \|f_\theta(q) - p_c\|^2, \qquad p_c(\theta) = \frac{1}{|S_c|} \sum_{x \in S_c} f_\theta(x), \tag{41}$$

where $f_\theta(\cdot)$ is the LSTM encoder that maps sequential traffic data into latent embeddings. Equation (40) enforces the classical prototypical-learning behavior: embeddings from the same class are pulled toward their prototype, while embeddings from different classes are pushed apart.

Differentiating (40) yields the local gradient update

$$\nabla_\theta L_i = 2(f_\theta(q) - p_{y_q}) \frac{\partial f_\theta(q)}{\partial \theta} - \frac{2}{|S_{y_q}|} \sum_{x \in S_{y_q}} (f_\theta(x) - p_{y_q}) \frac{\partial f_\theta(x)}{\partial \theta}. \tag{42}$$

This gradient simultaneously contracts intra-class embeddings and expands inter-class separation, enabling each device to refine its temporal feature extractor via

$$\theta_i^{t+1} = \theta_i^t - \eta \nabla_\theta L_i(\theta_i^t).$$

B. Effect of Structured Pruning and Quantization

Before model reuse or communication, Light-FFSL applies structured pruning and quantization, which modifies the update rule. Pruning projects parameters onto a reduced active subspace M via a projection operator $P_M(\cdot)$, while integer quantization applies the compression operator $C_\omega(\cdot)$ with fidelity ω:

$$\theta_i^{t+1} = C_\omega\big(P_M(\theta_i^t - \eta \nabla_\theta L_i(\theta_i^t))\big). \tag{43}$$

Both operators are non-expansive:

$$\|P_M(u) - P_M(v)\| \le \|u - v\|, \qquad \mathbb{E}\|C_\omega(\Delta) - \Delta\|^2 \le (1-\omega)\|\Delta\|^2.$$

Thus, they do not alter the order of convergence, but they act to (i) reduce the effective dimensionality of the optimization landscape via pruning and (ii) introduce bounded quantization noise proportional to $(1 - \omega)$. The combined effect yields a compressed learning manifold with lower variance and reduced memory footprint.

C. Adaptive and Collaborative Learning Behavior

Devices participate asynchronously based on energy, latency, and computational constraints. Dynamic scheduling increases the selection probability of devices with stronger gradient alignment ρ_i, leading to the aggregated gradient

$$g_t = \sum_{i \in S_t} \alpha_t^i \nabla_\theta L_i(\theta_t), \qquad \alpha_t^i = \frac{1}{m_t p_t^i},$$

where m_t is the number of participating devices in round t, and p_t^i is the probability of selecting device i.

Convergence-based update suppression restricts communication to rounds in which performance improves by at least τ, which introduces a bounded gradient perturbation:

$$\mathbb{E}\|g_t - \nabla f(\theta_t)\|^2 \le \bar{\tau}^2.$$

Together, dynamic scheduling and selective communication prevent divergence caused by data heterogeneity and unreliable devices, while ensuring that only informative gradients contribute to the global update.

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 23,2026 at 15:13:11 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3652009

TABLE I: Physical Meaning of Symbols in Theoretical Analysis

Symbol	Description
θ_t	Parameters of the LSTM encoder at communication round t.
$f_\theta(\cdot)$	LSTM mapping from sequential network traffic to latent embeddings.
$p_c(\theta)$	Prototype (mean embedding) of class c computed from its support set.
$L_i(\theta)$	Local few-shot episodic loss optimized on device i.
η	Learning rate controlling update step size.
M	Active parameter subspace after structured pruning.
$P_M(\cdot)$	Projection operator that enforces pruning mask (retains salient weights).
$C_\omega(\cdot)$	Quantization operator with fidelity ω (INT8 compression).
$\bar{\rho}$	Average gradient alignment among participating devices.
$\bar{\nu}$	Average gradient variance caused by non-IID (heterogeneous) data.
τ^2	Mean squared threshold for convergence-based communication control.
m_t	Number of devices selected in round t.
$p_{\min}$	Minimum participation probability ensuring fairness.
G^2	Upper bound on the squared gradient norm ensuring stability.
ω	Quantization fidelity factor; $\omega \approx 1$ implies near-lossless compression.

D. Overall Convergence Behavior

Theorem 1. Assume each $L_i(\theta)$ is L-smooth with bounded gradient variance σ^2 and heterogeneity δ^2. Suppose that device participation satisfies $m_t p_{\min} \geq c > 0$, and let ω denote quantization fidelity and $\bar{\tau}^2$ the communication-induced gradient noise. Then the sequence $\{\theta_t\}$ produced by the compressed update rule (43) satisfies

$$\frac{1}{T}\sum_{t=0}^{T-1} E\big[\|\nabla f_M(\theta_t)\|^2\big] \leq \frac{2\big(f(\theta_0) - f^*\big)}{\eta T} + O\Big((1-\bar{\rho})^2 + \frac{\bar{\nu}}{m_t p_{\min}} + \tau^2 + (1-\omega)\Big). \tag{44}$$

where f_M is the global objective restricted to the pruned subspace M and $\bar{\nu}$ is the effective variance term.

The first term governs the optimization progress, while the additive terms correspond to stationary errors arising from gradient misalignment, device heterogeneity, communication sparsity, and quantization noise.

Corollary 1. If the pruned subspace M retains sufficient representational capacity and quantization fidelity is high ($\omega \approx 1$), then all error terms in (44) vanish asymptotically. Consequently, the compressed LSTM–Prototypical Network converges at the same rate as its full-precision counterpart, while requiring significantly less computation and memory.

Discussion. Theorem 1 shows that Light-FFSL maintains stable learning despite compression and partial device participation. Pruning and quantization create a low-dimensional manifold that reduces variance, while dynamic scheduling ensures that well-aligned gradients dominate the update. Convergence-based suppression further limits communication without harming training stability. These mechanisms jointly explain the strong accuracy–efficiency trade-off observed in Section V.

V. Performance Evaluation

This section presents the performance evaluation of the proposed Light-FFSL framework in a simulated IoT environment. The evaluation uses widely adopted IoT security datasets: TON_IoT [41], and Bot_IoT [42], focusing on device participation and convergence analysis, model efficiency, and performance analysis.

A. Experimental Setup

To evaluate the proposed framework in a realistic IoT scenario, we emulated 50 resource-constrained IoT devices using Python-based multiprocessing. Each device operated as an independent process with constrained CPU, memory, and bandwidth, while lightweight TCP sockets facilitated communication under varying network conditions including latency and packet loss. A detailed summary of all configuration parameters and thresholds is presented in Table II.

TABLE II: Experimental Settings

Parameter	Setting
Device Emulation	Python 3.10 processes simulating resource-constrained IoT devices
Number of Devices	50 virtual IoT nodes
Device Configuration	CPU frequency: 1.5 GHz, RAM: 512 MB–1 GB
Network Conditions	Latency: 10–100 ms, Packet Loss: 0%–5%
Base Model	LSTM-based Prototypical Network
LSTM Hidden Units	64
Embedding Dimension	32
Model Compression	30% unstructured pruning, 8-bit quantization
Optimizer	Adam
Learning Rate	0.001
Batch Size	32 episodes per communication round
Device Selection Thresholds	Energy $>$ 30%, CPU availability $>$ 40%, Latency $<$ 100 ms, Uptime $>$ 80%
Local Update Condition	Upload if validation loss improves $\geq$ 0.5% or loss change $<$ 0.1% over 5 epochs
Global Stability Condition	Reduce device participation if global validation loss change $<$ 0.1% over 5 rounds
Dynamic Participation	Top 50% devices initially; top 30% after global model stabilizes
Maximum Rounds	300
Server Hardware	Intel Core i7-11700K CPU, 32 GB RAM, Ubuntu 22.04 LTS
Software Stack	PyTorch 2.0, TensorFlow Lite, Scikit-learn
Datasets	TON_IoT, Bot_IoT

B. Datasets

The evaluation of the proposed Light-FFSL framework was conducted using two publicly available IoT security datasets: TON_IoT and Bot_IoT.

1) Frequent/Evolving Split Details: We treat the top-m classes ranked by training frequency as $\mathcal{C}_{\text{freq}}$, while the remaining classes $\mathcal{C}_{\text{new}}$ are excluded from training and only appear during evaluation. This split reflects the generalized few-shot setting, where the model must adapt to previously unseen attack classes from only a few labeled examples. For comparison with conventional supervised baselines, we also consider a closed-set evaluation in which all classes are included during training and testing. In this setting, supervised models are trained with the full labeled dataset, whereas

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 23,2026 at 15:13:11 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3652009

our few-shot model remains constrained to using only a few labeled samples per class. This dual evaluation protocol ensures fairness by assessing supervised approaches in their standard regime, while also highlighting the label efficiency and adaptability of few-shot learning methods.

2) TON_IoT: The TON_IoT dataset [41] offers a comprehensive representation of modern IoT security threats, incorporating telemetry data from IoT sensors, operating system logs, network traffic captures, and system processes. In this work, we utilize only the network traffic subset. This subset contains nine types of attacks and provides 44 features extracted from heterogeneous IoT network activity. The dataset exhibits a moderate level of class imbalance, with attack traffic slightly exceeding normal traffic. Preprocessing included feature normalization, categorical encoding, and the removal of irrelevant or redundant attributes.

3) BoT_IoT: The BoT_IoT dataset [42], captures extensive network traffic generated in a controlled IoT environment. It includes both benign traffic and a wide range of attack scenarios. Each record is characterized by 35 features. A major challenge with BoT_IoT is its severe class imbalance, where attack samples vastly outnumber benign ones. To address this, random undersampling was performed to balance the classes during training. It reduces redundant flows, improves sensitivity to rare classes, and accelerates training by lowering computation and communication overhead. Unlike cluster-based or heuristic undersampling, which require additional computation for representative selection, it provides immediate efficiency gains while still mitigating extreme imbalance in BoT_IoT. Feature selection based on mutual information was used to retain the most relevant attributes, and feature normalization was applied to standardize input scales.

C. Numerical Results

In this section, we evaluate the proposed Light-FFSL framework in terms of device participation and convergence behavior. We also evaluate its performance in comparison with standard FFSL and Few-shot Intrusion Detection System (FS-IDS) [23].

1) Device Participation and Convergence Analysis: This section analyzes the impact of dynamic participation and convergence-based updates on IoT devices and federated training performance.

To analyze device participation behavior throughout training, Fig. 2 depicts a heatmap of device engagement across 300 communication rounds. Each row corresponds to an IoT device (indexed 0 to 49), and each column to a training round. The intensity of shading indicates the relative frequency of participation, with darker shades representing more frequent selection. This visualization highlights the dynamic scheduling behavior of Light-FFSL, wherein device participation is adaptively modulated based on real-time evaluation of their normalized suitability scores derived from the five aforementioned metrics. Unlike traditional FL schemes that assume uniform device engagement, Light-FFSL continuously evaluates the suitability of each device for participation, allowing it to dynamically balance energy consumption, computational load, and training contribution.

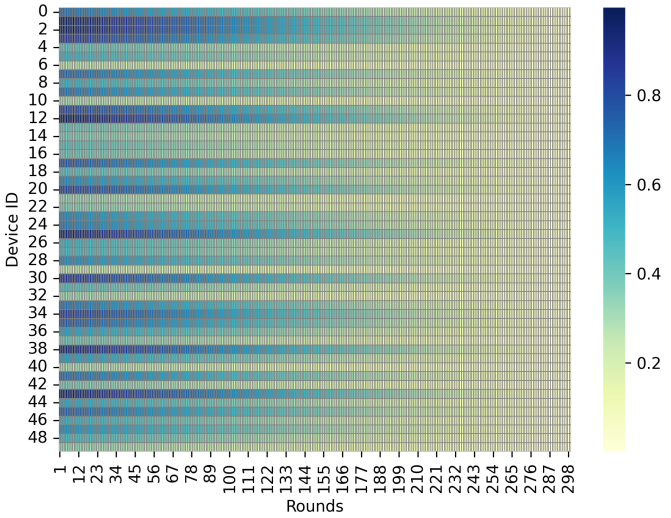


Fig. 2: Heatmap of participation frequency for 50 IoT devices over 300 communication rounds.

Fig. 3 compares Light-FFSL with standard FL in terms of device sustainability and convergence over 300 rounds. Light-FFSL maintains zero dead devices throughout training by leveraging dynamic scheduling, which adaptively selects participants based on real-time suitability scores. This prevents overuse of resource-constrained nodes, ensuring balanced participation. In contrast, standard FL's uniform participation depletes weaker devices early, leading to increasing device dropouts over time. Fig. 4 shows the global loss trends. Standard FL initially reduces loss quickly due to full participation, but its convergence slows as devices drop out, eventually plateauing. Light-FFSL, however, exhibits a smoother and more stable convergence trajectory. By prioritizing devices with higher convergence contributions, it accelerates learning and improves final performance. These results demonstrate the effectiveness of dynamic scheduling and convergence-based updates in sustaining device participation and optimizing model training, while avoiding unnecessary strain on less capable nodes.

2) Model Efficiency: To evaluate the efficiency of the proposed Light-FFSL framework in resource-constrained IoT environments, we analyze its performance in terms of model size and inference time. We compare Light-FFSL with the standard FFSL model and FS-IDS, along with two optimized variants: a pruned version and a pruned-and-quantized version.

As shown in Table III, the Light-FFSL (Pruned & Quantized) model is the most compact, requiring only 0.043 MB of storage, a reduction of 62.7% compared to FFSL and 66.9% compared to FS-IDS. Its inference latency is also the lowest at 0.65 seconds, representing nearly 80% improvement over both baselines. The pruned-only model achieves moderate gains, reducing size by up to 37.7% and latency to 2.66 seconds. These improvements stem from pruning, which removes low-contribution parameters and reduces complexity, and quantization, which compresses weights and activations into low-precision integer formats. Together, they provide substantial compression and speedup, confirming Light-FFSL's suitability

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 23,2026 at 15:13:11 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3652009

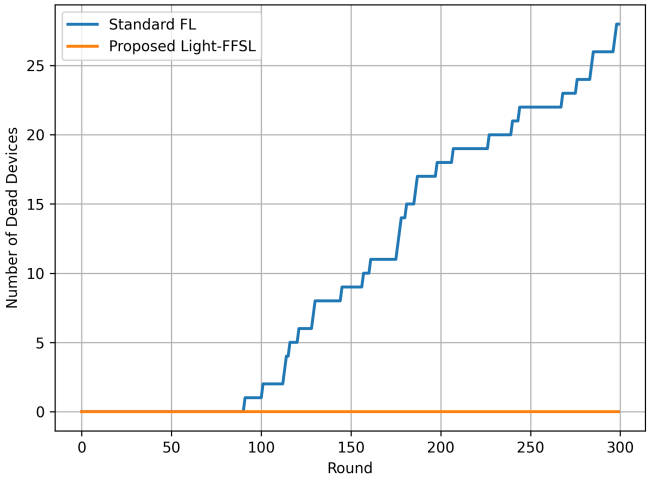


Fig. 3: Number of dead devices across federated rounds.

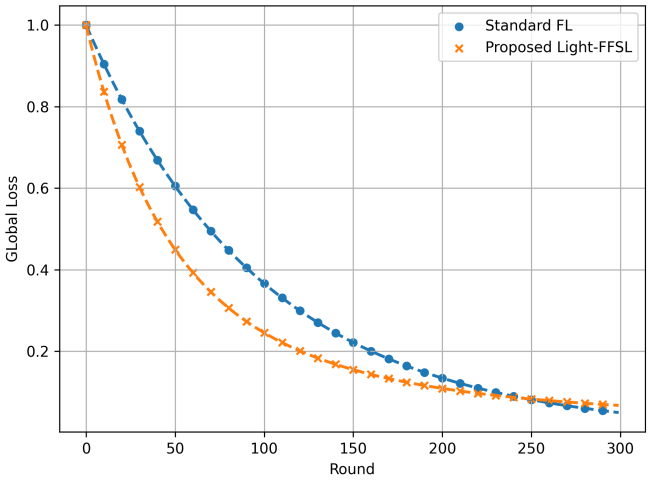


Fig. 4: Global loss over federated rounds.

for real-time deployment on IoT devices.

TABLE III: Model Size and Inference Time Comparison

Model	**Inference Time** (Seconds)	**Model Size** (MB)
FS-IDS	3.290	0.130
Standard FFSL	3.219	0.115
Light-FFSL (Pruned)	2.664	0.081
Light-FFSL (Pruned & Quantized)	0.654	0.043

3) Effect of Sequence Length on Model Performance: Table IV reports the weighted accuracy, precision, recall, and F1-score of the model across different sequence lengths T. The results show that model performance remains relatively consistent across $T = 15$ to $T = 30$, with only minor fluctuations in all metrics. While $T = 25$ yields the highest overall scores, the differences compared between shorter or longer sequences is marginal, suggesting that the choice of sequence length has a limited impact on detection capability. However, the computational complexity, measured in Multiply–Accumulate Operations (MACs), increases linearly with T, even though performance remains almost flat. This highlights the importance of considering both accuracy and complexity when selecting T, as longer sequences introduce higher computational cost without providing substantial performance improvements in resource-constrained IoT environments.

TABLE IV: Model Performance and Computational Complexity Across Varying Sequence Lengths (T)

T	**Accuracy**	**Precision**	**Recall**	**F1-score**	**MACs**
10	0.8141	0.8312	0.8141	0.8180	753,664
15	0.8681	0.8862	0.8681	0.8704	1,122,304
20	0.8647	0.8744	0.8647	0.8641	1,490,944
25	0.8860	0.8948	0.8860	0.8874	1,859,584
30	0.8746	0.8869	0.8746	0.8750	2,228,224

4) Performance Analysis on Evolving Attacks: To evaluate the performance of the proposed Light-FFSL framework, we compare it against the standard FFSL [43] and FS-IDS [23] on the ToN_IoT dataset across multiple (k, q) settings. As shown in Fig. 5, Light-FFSL consistently matches the performance of full-capacity FFSL while outperforming FS-IDS. Under the most demanding configuration $(k = 10, q = 30)$, Light-FFSL achieves an accuracy of 0.922 and an F1-score of 0.920, less than 1% below FFSL but up to 4.6% higher than FS-IDS. Similar trends hold across intermediate and constrained settings, where Light-FFSL maintains competitive accuracy and balanced F1-scores (e.g., 0.773 accuracy and 0.740 F1 at $(k = 3, q = 5)$), demonstrating stability even under limited supervision. On the Bot_IoT dataset, Light-FFSL again tracks closely with the standard FFSL, typically within 2%, while consistently outperforming FS-IDS. At $(k = 10, q = 30)$, it achieves 0.915 accuracy and 0.906 F1-score, reflecting only minor degradation from compression while still exceeding FS-IDS by about 1%. At intermediate and low-shot settings, the performance gap with FFSL remains small (e.g., 0.875 accuracy and 0.874 F1 at $(k = 5, q = 15)$, and 0.840 accuracy and 0.809 F1 at $(k = 3, q = 5)$), confirming that Light-FFSL sustains robust detection capability across diverse configurations. The modest reduction in Light-FFSL's performance stems mainly from quantization, which compresses weights and activations into 8-bit integers. While this slightly impacts accuracy, it greatly reduces memory footprint and inference latency. Combined with structured pruning, the performance drop remains below 2% compared to standard FFSL, yet Light-FFSL still achieves a consistent 3–5% improvement over FS-IDS. This minimal efficiency–accuracy trade-off confirms Light-FFSL as a practical and scalable solution for real-time intrusion detection in resource-constrained IoT environments.

5) Comparison with Existing NIDS Approaches: To ensure a fair evaluation, all models are compared using only seen attack classes, thereby eliminating biases that could arise from differing treatment of unseen threats across methods. Table V summarizes the performance of Light-FFSL against several NIDS approaches. The results show that Light-FFSL achieves a competitive accuracy of 97.5% with balanced precision, recall, and F1-score, while maintaining an exceptionally lightweight design (0.043 MB) and low inference latency (0.654 s). In contrast, methods such as GMCWAE and NIDS-CNNRF achieve strong detection performance but at the

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 23,2026 at 15:13:11 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3652009

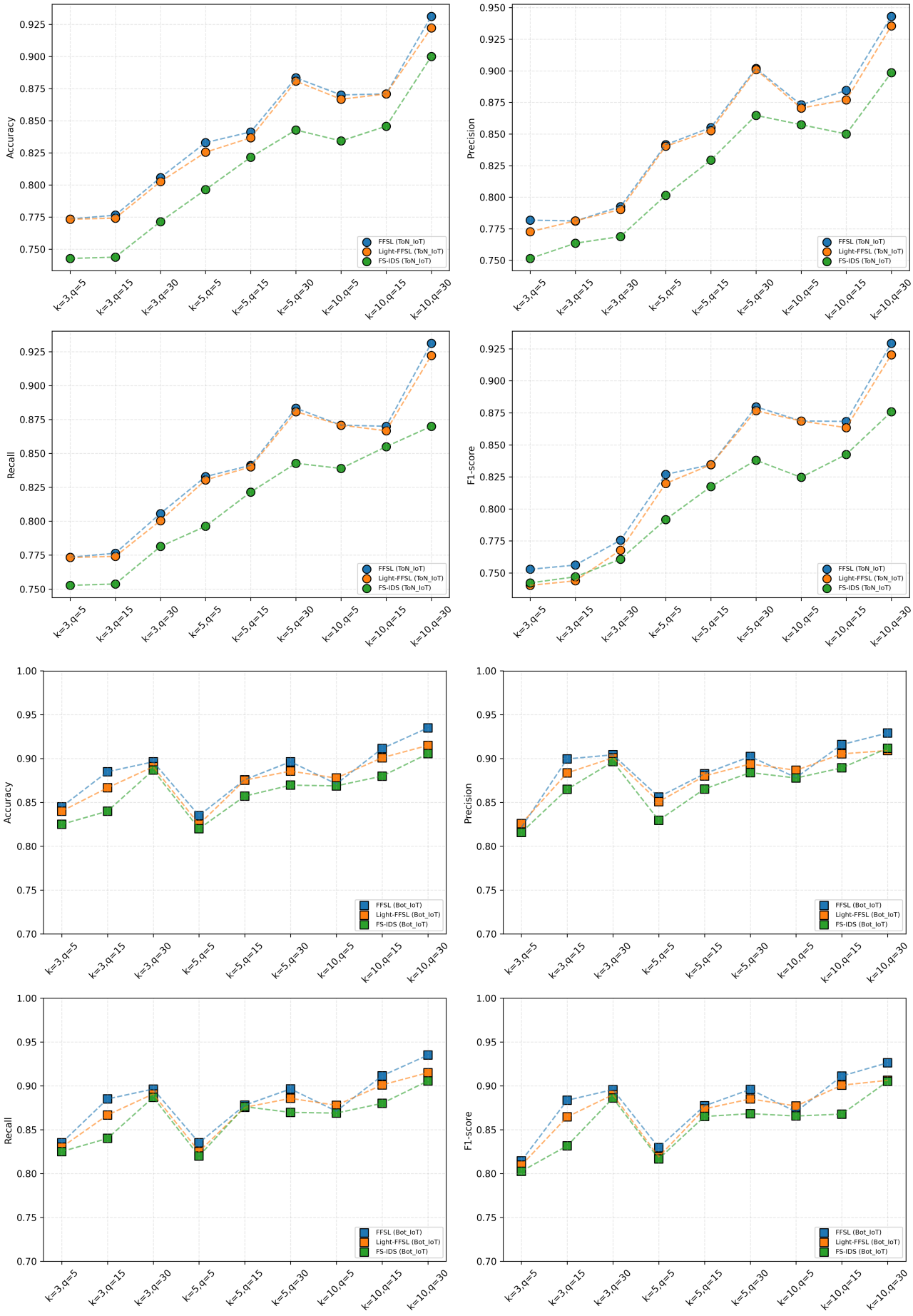


Fig. 5: Comparison of Light-FFSL, FFSL, and FS-IDS on ToN_IoT and bot_IoT datasets.

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 23,2026 at 15:13:11 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3652009

TABLE V: Comparison of Light-FFSL with Existing NIDS Approaches

Model	Accuracy	Precision	Recall	F1-score	Model Size (MB)	Inference Time (s)
GMCWAE [21]	0.9423	0.9444	0.9826	0.9635	1.202	2.299
A-NIDS [24]	0.9414	0.9963	0.9261	0.9602	0.345	0.993
NIDS-CNNRF [20]	0.9825	0.9974	0.9802	0.9883	193.622	4.702
ANN [35]	0.9056	0.9077	0.9056	0.9064	9.013	0.392
Light-FFSL (Proposed)	0.9753	0.9772	0.9752	0.9758	0.043	0.654

cost of significantly larger model sizes and higher inference times, making them less suitable for resource-constrained IoT devices. Likewise, A-NIDS and ANN provide reasonable accuracy, yet their larger footprints limit their practicality for real-time deployment. Overall, this consistent and fair evaluation highlights that Light-FFSL achieves the best balance between detection accuracy, computational efficiency, and deployment feasibility, demonstrating its suitability for securing resource-constrained IoT environments.

VI. Conclusion

This paper proposed Light-FFSL, a lightweight federated few-shot learning framework designed to address the critical challenges of intrusion detection in resource-constrained IoT environments. By integrating dynamic scheduling, convergence-based updates, and a prune-and-quantize LSTM-based prototypical network, the proposed system effectively balances computational efficiency, data privacy, and learning with minimal labeled data. Extensive experimental evaluation across two benchmark datasets (ToN_IoT and Bot_IoT) demonstrated that Light-FFSL not only achieves competitive detection performance compared to full-capacity models but also significantly reduces model size and inference latency, making it highly suitable for low-power IoT deployment. Moreover, the framework maintains device longevity and ensures sustainable participation across FL rounds through adaptive device selection. These results show that Light-FFSL is a practical and scalable solution for securing IoT networks under stringent resource and privacy constraints.

References

[1] W. Mao, Z. Zhao, Z. Chang, G. Min, and W. Gao, "Energy-efficient industrial internet of things: Overview and open issues," *IEEE transactions on industrial informatics*, vol. 17, no. 11, pp. 7225–7237, 2021.

[2] E. Baccour, N. Mhaisen, A. A. Abdellatif, A. Erbad, A. Mohamed, M. Hamdi, and M. Guizani, "Pervasive ai for iot applications: A survey on resource-efficient distributed artificial intelligence," *IEEE Communications Surveys & Tutorials*, vol. 24, no. 4, pp. 2366–2418, 2022.

[3] S. S. Shafin, G. Karmakar, and I. Mareels, "Obfuscated memory malware detection in resource-constrained iot devices for smart city applications," *Sensors*, vol. 23, no. 11, p. 5348, 2023.

[4] L. Babun, K. Denney, Z. B. Celik, P. McDaniel, and A. S. Uluagac, "A survey on iot platforms: Communication, security, and privacy perspectives," *Computer Networks*, vol. 192, p. 108040, 2021.

[5] I. Butun, P. Österberg, and H. Song, "Security of the internet of things: Vulnerabilities, attacks, and countermeasures," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 1, pp. 616–644, 2019.

[6] C. Xenofontos, I. Zografopoulos, C. Konstantinou, A. Jolfaei, M. K. Khan, and K.-K. R. Choo, "Consumer, commercial, and industrial iot (in) security: Attack taxonomy and case studies," *IEEE Internet of Things Journal*, vol. 9, no. 1, pp. 199–221, 2021.

[7] S. A. Abdulkareem, C. H. Foh, M. Shojafar, F. Carrez, and K. Moessner, "Network intrusion detection: An iot and non iot-related survey," *IEEE Access*, 2024.

[8] O. H. Abdulganiyu, T. A. Tchakoucht, and Y. K. Saheed, "Towards an efficient model for network intrusion detection system (ids): systematic literature review," *Wireless Networks*, vol. 30, no. 1, pp. 453–482, 2024.

[9] B. Nawaal, U. Haider, I. U. Khan, and M. Fayaz, "Signature-based intrusion detection system for iot," in *Cyber Security for Next-Generation Computing Technologies*, pp. 141–158, CRC Press, 2024.

[10] S. Walling and S. Lodh, "An extensive review of machine learning and deep learning techniques on network intrusion detection for iot," *Transactions on Emerging Telecommunications Technologies*, vol. 36, no. 2, p. e70064, 2025.

[11] N. A. Khalek, N. Abdolkhani, and W. Hamouda, "Deep reinforcement learning for joint power control and access coordination in energy harvesting ciot," *IEEE Internet of Things Journal*, 2024.

[12] M. El Tanab and W. Hamouda, "Resource allocation for underlay cognitive radio networks: A survey," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 2, pp. 1249–1276, 2016.

[13] N. Chaabouni, M. Mosbah, A. Zemmari, C. Sauvignac, and P. Faruki, "Network intrusion detection for iot security based on learning techniques," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 3, pp. 2671–2701, 2019.

[14] K. He, D. D. Kim, and M. R. Asghar, "Adversarial machine learning for network intrusion detection systems: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 1, pp. 538–566, 2023.

[15] T. Al-Shurbaji, M. Anbar, S. Manickam, I. H. Hasbullah, N. ALfriehate, B. A. Alabsi, A. R. Alzighaibi, and H. Hashim, "Deep learning-based intrusion detection system for detecting iot botnet attacks: a review," *IEEE Access*, 2025.

[16] N. A. Khalek, D. H. Tashman, and W. Hamouda, "Advances in machine learning-driven cognitive radio for wireless networks: A survey," *IEEE Communications Surveys & Tutorials*, vol. 26, no. 2, pp. 1201–1237, 2023.

[17] E. Benkhelifa, T. Welsh, and W. Hamouda, "A critical review of practices and challenges in intrusion detection systems for iot: Toward universal and resilient systems," *IEEE communications surveys & tutorials*, vol. 20, no. 4, pp. 3496–3509, 2018.

[18] E. Gyamfi and A. D. Jurcut, "Novel online network intrusion detection system for industrial iot based on oi-svdd and as-elm," *IEEE Internet of Things Journal*, vol. 10, no. 5, pp. 3827–3839, 2022.

[19] R. Zhao, G. Gui, Z. Xue, J. Yin, T. Ohtsuki, B. Adebisi, and H. Gacanin, "A novel intrusion detection method based on lightweight neural network for internet of things," *IEEE Internet of Things Journal*, vol. 9, no. 12, pp. 9960–9972, 2021.

[20] K. Yang, J. Wang, G. Zhao, X. Wang, W. Cong, M. Yuan, J. Luo, X. Dong, J. Wang, and J. Tao, "Nids-cnnrf integrating cnn and random forest for efficient network intrusion detection model," *Internet of Things*, p. 101607, 2025.

[21] D. Bian and J. Liu, "Gmcwae: A representation learning technique for network intrusion detection in iot," *IEEE Internet of Things Journal*, 2025.

[22] J. Tong and Y. Zhang, "A real-time label-free self-supervised deep learning intrusion detection for handling new type and few-shot attacks in iot networks," *IEEE Internet of Things Journal*, 2024.

[23] Y. Ouyang, B. Li, Q. Kong, H. Song, and T. Li, "Fs-ids: a novel few-shot learning based intrusion detection system for scada networks," in *ICC 2021-IEEE International Conference on Communications*, pp. 1–6, IEEE, 2021.

[24] C. Zha, Z. Wang, Y. Fan, B. Bai, Y. Zhang, S. Shi, and R. Zhang, "A-nids: adaptive network intrusion detection system based on clustering and stacked ctgan," *IEEE Transactions on Information Forensics and Security*, 2025.

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 23,2026 at 15:13:11 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3652009

[25] C. Wu, J. Sun, J. Chen, M. Alazab, Y. Liu, and Y. Xiang, "Tcg-ids: Robust network intrusion detection via temporal contrastive graph learning," *IEEE Transactions on Information Forensics and Security*, 2025.

[26] O. Friha, M. A. Ferrag, L. Shu, L. Maglaras, K.-K. R. Choo, and M. Nafaa, "Felids: Federated learning-based intrusion detection system for agricultural internet of things," *Journal of Parallel and Distributed Computing*, vol. 165, pp. 17–31, 2022.

[27] P. Bhale, D. R. Chowdhury, S. Biswas, and S. Nandi, "Optimist: lightweight and transparent ids with optimum placement strategy to mitigate mixed-rate ddos attacks in iot networks," *IEEE Internet of Things Journal*, vol. 10, no. 10, pp. 8357–8370, 2023.

[28] R. Zhao, Y. Wang, Z. Xue, T. Ohtsuki, B. Adebisi, and G. Gui, "Semisupervised federated-learning-based intrusion detection method for internet of things," *IEEE Internet of Things Journal*, vol. 10, no. 10, pp. 8645–8657, 2022.

[29] J. Zhang, C. Luo, M. Carpenter, and G. Min, "Federated learning for distributed iiot intrusion detection using transfer approaches," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 7, pp. 8159–8169, 2022.

[30] R. Benameur, A. Dahane, S. Souihi, and A. Mellouk, "A novel federated learning based intrusion detection system for iot networks," in *ICC 2024-IEEE International Conference on Communications*, pp. 2402–2407, IEEE, 2024.

[31] A. Aljuhani, A. Alamri, P. Kumar, and A. Jolfaei, "An intelligent and explainable saas-based intrusion detection system for resource-constrained iomt," *IEEE Internet of Things Journal*, 2023.

[32] F. Naeem, M. Ali, and G. Kaddoum, "Federated-learning-empowered semi-supervised active learning framework for intrusion detection in zsm," *IEEE Communications Magazine*, vol. 61, no. 2, pp. 88–94, 2023.

[33] M. Abdel-Basset, H. Hawash, R. K. Chakrabortty, and M. J. Ryan, "Semi-supervised spatiotemporal deep learning for intrusions detection in iot networks," *IEEE Internet of Things Journal*, vol. 8, no. 15, pp. 12251–12265, 2021.

[34] W. Yao, H. Peng, Q. Li, and X. Shen, "Modeling realistic adversarial traffic against deep learning-based intrusion detection system in industrial iot," *IEEE Internet of Things Journal*, 2025.

[35] A. Farhat, A. Eldosouky, M. Ibnkahla, and A. Matrawy, "Interaction-aware trust management scheme for iot systems with machine learning-based attack detection," *IEEE Internet of Things Journal*, 2025.

[36] D. C. Nguyen, M. Ding, P. N. Pathirana, A. Seneviratne, J. Li, and H. V. Poor, "Federated learning for internet of things: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 3, pp. 1622–1658, 2021.

[37] W. Y. B. Lim, N. C. Luong, D. T. Hoang, Y. Jiao, Y.-C. Liang, Q. Yang, D. Niyato, and C. Miao, "Federated learning in mobile edge networks: A comprehensive survey," *IEEE communications surveys & tutorials*, vol. 22, no. 3, pp. 2031–2063, 2020.

[38] Z. Liu, J. Guo, W. Yang, J. Fan, K.-Y. Lam, and J. Zhao, "Privacy-preserving aggregation in federated learning: A survey," *IEEE Transactions on Big Data*, 2022.

[39] Y. He and L. Xiao, "Structured pruning for deep convolutional neural networks: A survey," *IEEE transactions on pattern analysis and machine intelligence*, vol. 46, no. 5, pp. 2900–2919, 2023.

[40] H. Wu, P. Judd, X. Zhang, M. Isaev, and P. Micikevicius, "Integer quantization for deep learning inference: Principles and empirical evaluation," *arXiv preprint arXiv:2004.09602*, 2020.

[41] T. M. Booij, I. Chiscop, E. Meeuwissen, N. Moustafa, and F. T. Den Hartog, "Ton_iot: The role of heterogeneity and the need for standardization of features and attack types in iot network intrusion data sets," *IEEE Internet of Things Journal*, vol. 9, no. 1, pp. 485–496, 2021.

[42] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-iot dataset," *Future Generation Computer Systems*, vol. 100, pp. 779–796, 2019.

[43] A. Saleem and W. Hamouda, "Federated few-shot learning for robust and privacy-driven network intrusion detection in iot," in *ICC 2025-IEEE International Conference on Communications*, pp. 2949–2954, IEEE, 2025.

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 23,2026 at 15:13:11 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted,
but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.